

On s'intéresse à une application de traitement de notes et d'analyse de textes en Python.

Partie 1 : Traitement de notes (3 points)

On considère la liste des notes suivantes :

notes = [12, 15, 8, 19, 14, 7]

1. Écrire une fonction `moyenne(l)` qui retourne la moyenne des éléments d'une liste `l`.
2. Écrire une fonction `sup_moyenne(l)` qui retourne la liste des notes supérieures ou égales à la moyenne, en utilisant `filter` et une fonction `lambda`.
3. Écrire une fonction `applique_bonus(l, f)` qui prend en paramètre une liste `l` et une fonction `f`, et retourne une nouvelle liste où chaque note a été modifiée par `f`.

Exemple : ajouter 2 points à toutes les notes.

Partie 2 : Récursion et suites (2 points)

1. Écrire une fonction récursive `fact(n)` qui calcule la factorielle d'un entier $n \geq 0$.
2. Écrire une fonction récursive `fibonacci(n)` qui retourne la liste des `n` premiers termes de la suite de Fibonacci.
3. Écrire une fonction `fibonacci_pairs(n)` qui utilise `filter` pour ne garder que les nombres pairs de la suite de Fibonacci.

Partie 3 : Analyse de textes (1 point)

On considère le texte suivant :

texte = "NSI est une matière intéressante. NSI utilise Python pour apprendre la programmation."

1. Écrire une fonction `compte_mots(t)` qui retourne le nombre total de mots dans le texte `t`.
2. Écrire une fonction `compte_mot_cle(t, mot)` qui retourne combien de fois le mot `mot` apparaît dans le texte `t`, en utilisant `filter` et une fonction `lambda`.
3. Écrire une fonction `mots_longueur(t, n)` qui retourne la liste des mots dont la longueur est supérieure ou égale à `n`.

Corrigé

Partie 1 : Traitement de notes

```
notes = [12, 15, 8, 19, 14, 7]
```

```
# 1. Moyenne
```

```
def moyenne(l):  
    if len(l) == 0:  
        return 0  
    return sum(l) / len(l)
```

```
# Test
```

```
print("Moyenne :", moyenne(notes)) # 12.5
```

```
# 2. Notes supérieures ou égales à la moyenne
```

```
def sup_moyenne(l):  
    m = moyenne(l)  
    return list(filter(lambda x: x >= m, l))
```

```
# Test
```

```
print("Notes >= moyenne :", sup_moyenne(notes)) # [15, 19, 14]
```

```
# 3. Appliquer un bonus à toutes les notes
```

```
def applique_bonus(l, f):  
    return list(map(f, l))
```

```
# Exemple : bonus de 2 points
```

```
print("Notes avec bonus :", applique_bonus(notes, lambda x: x + 2)) # [14, 17, 10, 21, 16, 9]
```

Partie 2 : Récursion et suites

```
# 1. Factorielle
```

```
def fact(n):  
    if n == 0 or n == 1:  
        return 1  
    return n * fact(n - 1)
```

```
# Test
```

```
print("5! =", fact(5)) # 120
```

```
# 2. Fibonacci (liste des n premiers termes)
```

```
def fibonacci(n):  
    if n == 0:  
        return []  
    elif n == 1:  
        return [0]  
    elif n == 2:  
        return [0, 1]  
    else:  
        fibs = fibonacci(n - 1)  
        fibs.append(fibs[-1] + fibs[-2])  
        return fibs
```

```
# Test
print("Fibonacci 7 termes :", fibonacci(7)) # [0, 1, 1, 2, 3, 5, 8]
```

```
# 3. Fibonacci pairs
def fibonacci_pairs(n):
    return list(filter(lambda x: x % 2 == 0, fibonacci(n)))
```

```
# Test
print("Fibonacci pairs :", fibonacci_pairs(7)) # [0, 2, 8]
```

Partie 3 : Analyse de textes

texte = "NSI est une matière intéressante. NSI utilise Python pour apprendre la programmation."

```
# 1. Compter les mots
def compte_mots(t):
    return len(t.split())
```

```
# Test
print("Nombre de mots :", compte_mots(texte)) # 11
```

```
# 2. Compter les occurrences d'un mot clé
def compte_mot_cle(t, mot):
    mots = t.split()
    return len(list(filter(lambda x: x.strip("!.?") == mot, mots)))
```

```
# Test
print("Occurrences de 'NSI' :", compte_mot_cle(texte, "NSI")) # 2
```

```
# 3. Mots de longueur >= n
def mots_longueur(t, n):
    mots = t.split()
    return list(filter(lambda x: len(x.strip("!.?")) >= n, mots))
```

```
# Test
print("Mots >= 5 lettres :", mots_longueur(texte, 5))
# ['matière', 'intéressante', 'utilise', 'Python', 'apprendre', 'programmation']
```